

Lista de Exercícios — Filas (C)

Estrutura base (vista em aula):

```
#define MAX_ELEMENTOS 100

struct fila {
    int elementos[MAX_ELEMENTOS];
    int primeiro;
    int ultimo;
};
```

Operações já implementadas em aula (você pode reutilizá-las): `cria`, `enfilera`, `desenfilera`, `tamanho`, `destrói`, além das versões que tratam fila cheia e fila vazia e do arranjo tratado como círculo (a posição seguinte a `MAX_ELEMENTOS - 1` é a posição 0).

Regra geral da lista: salvo indicação em contrário, você não pode acessar o vetor `elementos` nem os campos `primeiro`/`ultimo` diretamente fora das funções da própria fila. Use apenas as operações da interface (`enfilera`, `desenfilera`, `tamanho`, etc.). Lembre-se: numa fila, o primeiro a entrar é o primeiro a sair (FIFO).

Nível 1 — Aquecendo com a interface

1.1 — Programa de teste

Escreva uma função `main` que: cria uma fila, enfileira os valores 3, 7, 1, 9, imprime o tamanho, desenfileira e imprime todos os elementos (um por linha) e, por fim, destrói a fila. Confirme na saída que a ordem de remoção é 3, 7, 1, 9 (o primeiro a entrar é o primeiro a sair).

1.2 — frente (espiar sem remover)

Implemente

```
int frente(struct fila *f);
```

que devolve o elemento do início da fila sem desenfileirá-lo. A fila deve continuar exatamente igual depois da chamada. Trate o caso de fila vazia.

1.3 — esta_vazia e esta_cheia

Implemente duas funções que devolvem 1 (verdadeiro) ou 0 (falso):

```
int esta_vazia(struct fila *f);
int esta_cheia(struct fila *f);
```

Depois, reescreva as suas versões de `enfilera` e `desenfilera` usando essas funções em vez de comparar os índices "na mão".

1.4 — esvazia

Implemente `void esvazia(struct fila *f);` que remove todos os elementos da fila (deixando-a vazia) sem destruí-la. Ela deve poder ser usada normalmente depois.

1.5 — Imprimir sem destruir

Implemente `void imprime(struct fila *f);` que imprime os elementos do início até o fim, sem alterar a fila ao final. Cuidado: como o arranjo é circular, o `ultimo` pode estar "antes" do `primeiro` no vetor.

Nível 2 — Manipulando filas

2.1 — Copiar uma fila

Implemente `struct fila *copia(struct fila *f)`; que devolve uma nova fila com os mesmos elementos, na mesma ordem, sem modificar a original.

2.2 — Duas filas são iguais?

Implemente `int iguais(struct fila *a, struct fila *b)`; que devolve verdadeiro se as duas filas têm os mesmos elementos, na mesma ordem. Ao final, `a` e `b` devem permanecer intactas.

2.3 — Inverter uma fila

Implemente `void inverte(struct fila *f)`; que inverte a ordem dos elementos da fila (o que estava no fim passa para o início). Dica: uma pilha auxiliar resolve isso de forma natural.

2.4 — Intercalar duas filas

Implemente `struct fila *intercala(struct fila *a, struct fila *b)`; que devolve uma nova fila com os elementos de `a` e `b` alternados (`a0, b0, a1, b1, ...`). Se uma acabar antes, o resto da outra vai no final.

2.5 — Remover todas as ocorrências de um valor

Implemente `void remove_valor(struct fila *f, int v)`; que remove todas as ocorrências de `v` mantendo a ordem dos demais elementos. Use apenas operações de fila (dica: uma fila auxiliar).

Nível 3 — Aplicações clássicas

3.1 — Simulação de atendimento (fila do banco)

Cada cliente é representado por um inteiro: o tempo (em minutos) que leva para ser atendido. Leia vários clientes, enfileire-os e simule o atendimento em ordem de chegada, calculando o tempo médio de espera (quanto cada cliente esperou na fila antes de começar a ser atendido).

3.2 — Números binários de 1 a N

Usando apenas uma fila, gere e imprima as representações binárias dos números de 1 até `N`. Ideia: enfileire "1"; a cada passo, desenfileire uma string `s`, imprima-a e enfileire `s + "0"` e `s + "1"`.

- `N = 5` → 1, 10, 11, 100, 101

3.3 — Batata quente (problema de Josephus)

`N` crianças em roda (numeradas de 1 a `N`) passam uma batata. A cada `k` passagens, quem está com a batata sai da roda. Usando uma fila circular (desenfileira e enfileira de volta `k-1` vezes, depois elimina o próximo), imprima a ordem de eliminação e quem sobra por último.

3.4 — Escalonamento round-robin

Você tem processos, cada um com um tempo total de CPU. Simule um escalonador round-robin com fatia de tempo (quantum) `q`: cada processo roda por até `q`, e se ainda tiver trabalho volta para o fim da fila. Imprima a ordem em que os processos terminam.

3.5 — Fila circular na prática

Implemente `enfileira` e `desenfileira` tratando corretamente a "volta" do índice (quando ele passa de `MAX_ELEMENTOS - 1` volta para 0) e o problema de distinguir fila cheia de fila vazia — as duas situações deixam `primeiro == ultimo`. Explique, em comentário, qual estratégia você escolheu (sacrificar uma posição do vetor ou guardar um contador de elementos).

Nível 4 — Desafios

4.1 — Fila usando duas pilhas

Implemente uma fila usando apenas duas pilhas (`entrada` e `saida`) e as operações de pilha. Garanta a ordem FIFO nas operações `enfileira` e `desenfileira`. Qual o custo amortizado de cada operação?

4.2 — Fila com ponteiros (lista encadeada)

Reimplemente a fila usando nós encadeados em vez de arranjo, mantendo exatamente a mesma interface (`cria`, `enfilera`, `desenfilera`, `tamanho`, `destroi`). Sem `MAX_ELEMENTOS`: a fila cresce enquanto houver memória. Guarde ponteiros para o primeiro e o último nó para que ambas as operações sejam $O(1)$.

```
struct no {
    int dado;
    struct no *prox;
};

struct fila {
    struct no *primeiro;
    struct no *ultimo;
    int tamanho;
};
```

Pergunta para responder junto: por que guardar o ponteiro para o `ultimo` importa aqui? Essa versão ainda precisa testar "fila cheia"?

4.3 — Deque (fila dupla)

Implemente uma fila de duas pontas (deque), na qual é possível inserir e remover tanto no início quanto no fim: `insere_inicio`, `insere_fim`, `remove_inicio`, `remove_fim`. Faça com arranjo circular ou com ponteiros (sua escolha) e comente a complexidade de cada operação.

4.4 — Fila genérica (void *)

A partir da sua fila com ponteiros, generalize os nós para guardar `void *dado`, de modo que a mesma fila possa armazenar `int`, `double`, `struct`, etc. Assinaturas:

```
void enfilera(struct fila *f, void *dado);
void *desenfilera(struct fila *f);
```

Escreva um `main` que enfileire alguns `double` alocados dinamicamente e depois os recupere corretamente (atenção ao `cast` e ao `free`).

4.5 — Fila de prioridade simples

Implemente uma fila em que cada elemento tem uma prioridade inteira, e `desenfilera` sempre remove o elemento de maior prioridade (empates saem por ordem de chegada). Comente a complexidade da sua solução e como ela poderia melhorar.

Para todos os exercícios

Para cada função que você implementar, responda:

- Ela funciona corretamente quando a fila está vazia?
- Ela funciona quando a fila está cheia (na versão com arranjo circular)?
- Ela trata corretamente a volta do índice no arranjo circular?
- Há algum vazamento de memória (`malloc` sem `free`)?